

2.Filtering, Sorting,Calculating Data

February 19, 2025

0.0.1 Filtering Data

WHERE Clause Operators

```
SELECT column_name, column_name
FROM table_name
WHERE column_name operator value;
```

Operator	Description
=	Equal
<>	Not equal. Note: In some versions of SQL this operator may be written as !=
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
BETWEEN	Between an inclusive range
IS NULL	Is a null value

Filtering with a Range of Values

```
SELECT ProductName
,UnitPrice
,SupplierID
,UnitsInStock
FROM Products
WHERE UnitsInStock BETWEEN 15
AND 80;
```

	ProductName	UnitPrice	SupplierID	UnitsInStock
1	Chai	18	1	39
2	Chang	19	1	17
3	Chef Anton's Cajun Seasoning	22	2	53
4	Uncle Bob's Organic Dried Pears	30	3	15
5	Mishi Kobe Niku	97	4	29
6	Bkura	31	4	31
7	Queso Cabrales	21	5	22
8	Konbu	6	6	24
9	Tofu	23.25	6	35
10	Genen Shouyu	15.5	6	39
11	Pavlova	17.45	7	29
12	Carnarvon Tigers	62.5	7	42
13	Teatime Chocolate Biscuits	9.2	8	25

IN Operator Example

```
SELECT
ProductID
,UnitPrice
,SupplierID
From Products
WHERE SupplierID IN (9, 10,
11);
```

	ProductID	UnitPrice	SupplierID
1	22	21	9
2	23	9	9
3	24	4.5	10
4	25	14	11
5	26	31.23	11
6	27	43.9	11

OR Operator

DBMS will not evaluate the second conditions in a WHERE clause if the first condition is met

Use for any rows matching the specific conditions

	ProductID	UnitPrice	SupplierID	ProductName
1	14	23.25	6	Tofu

```
SELECT
ProductName
,ProductID
,UnitPrice
,SupplierID
,ProductName
From Products
Where ProductName = 'Tofu' OR
'Konbu';
```

OR with AND

	ProductID	UnitPrice	SupplierID
1	22	21	9
2	23	9	9
3	26	31.23	11
4	27	43.9	11

```
SELECT
ProductID
,UnitPrice
,SupplierID
FROM Products
WHERE SupplierID = 9 OR
SupplierID = 11
AND UnitPrice > 15;
```

	ProductID	UnitPrice	SupplierID
1	22	21	9
2	26	31.23	11
3	27	43.9	11

```
SELECT
ProductID
,UnitPrice
,SupplierID
FROM Products
WHERE (SupplierID = 9 OR
SupplierID = 11)
AND UnitPrice > 15;
```

NOT Operator

	EmployeeID	LastName	FirstName	Title	TitleOfCourtesy	Birth
1	2	Fuller	Andrew	Vice President, Sales	Dr.	2/19/1952
2	3	Leverling	Janet	Sales Representative	Ms.	8/30/1963
3	4	Peacock	Margaret	Sales Representative	Mrs.	9/19/1937

```
SELECT *  
FROM Employees  
WHERE NOT City='London' AND  
NOT City='Seattle';
```

What Are Wildcards?

Special character used to match parts of a value

Search pattern made from literal text, wildcard character, or a combination

Uses LIKE as an operator (though technically a predicate)

Using % Wildcards

Wildcard	Action
'%Pizza'	Grabs anything ending with the word pizza
'Pizza%'	Grabs anything after the word pizza
'%Pizza%'	Grabs anything before and after the word pizza

Using % Wildcards

Wildcard	Action
'S%E'	Grabs anything that starts with "S" and ends with "E" (Like Sadie)
't%@gmail.com'	Grabs gmail addresses that start with "t" (hoping to find Tom)

0.0.2 Sorting Data

Why Sort Data?

Sorting data logically helps keep information you want on top

ORDER BY clause allows user to sort data by particular columns

```
SELECT something  
FROM database  
ORDER BY characteristic
```

0.0.3 Calculating & Functions

Multiplication Example

Total units on order multiplied by the unit price to calculate the total order cost

```
SELECT
ProductID
,UnitsOnOrder
,UnitPrice
,UnitsOnOrder * UnitPrice AS Total_Order_Cost
FROM Products
```

Combining Math Operations

Unit price, minus my discount, divided by the quantity

```
SELECT
ProductID
,Quantity
,UnitPrice
,Discount
,(UnitPrice - Discount)*Quantity AS
Total_Cost
FROM OrderDetails;
```

Aggregate Functions

Function	Description
AVG ()	Averages a column of values
COUNT ()	Counts the number of values
MIN ()	Finds the minimum value
MAX ()	Finds the maximum value
SUM ()	Sums the column values

AVERAGE Function

Rows containing NULL values are ignored by the AVERAGE function

```
SELECT AVG(UnitPrice) AS avg_price
FROM products
```

COUNT Function

COUNT (*) – Counts all the rows in a table containing values or NULL values

```
SELECT COUNT (*) AS
total_customers
FROM Customers;
```

Count (column) – Counts all the rows in a specific column ignoring NULL values

```
SELECT COUNT(CustomerID) AS
total_customers
FROM Customers
```

MAX and MIN Functions

NULL values are ignored by the MIN and MAX functions

```
SELECT MAX(UnitPrice) AS max_prod_price
FROM Products
```

```
SELECT MAX(UnitPrice) AS max_prod_price
,MIN(UnitPrice) AS min_prod_price
FROM Products
```


SUM Aggregate Function

```
SELECT SUM(UnitPrice) AS  
       total_prod_price  
FROM Products
```

```
SELECT SUM(UnitPrice*UnitsInStock)  
       AS total_price  
FROM Products  
WHERE SupplierID = 23;
```

Using D Aggreg

If DISTINCT is
assumed

Cannot use D

No value to u
functions

```
SELECT CO  
FROM Cust
```

Grouping Example

Counts customers after group on region rather than counting the whole table

```
SELECT
  Region
, COUNT(CustomerID) AS total_customers
FROM Customers
GROUP BY Region;
```

Additional GROUP BY Information

GROUP BY clauses can contain multiple columns

Every column in your SELECT statement must be present in a GROUP BY clause, except for aggregated calculations

NULLs will be grouped together if your GROUP BY column contains NULLs

Grouping Example

```
SELECT  
CustomerID  
,COUNT (*) AS orders  
FROM Orders  
GROUP BY CustomerID  
HAVING COUNT (*) >=2;
```

WHERE vs. HAVING

WHERE filters before data is grouped

HAVING filters after data is grouped

Rows eliminated by the WHERE clause will not be included in the group